



机器学习与 Python 实践

第六次作业

基于集成学习的贷款申请审批预测模型

姓名：

专业： 强基班

学号：

教师：

学院： 信息科学与工程学院

2026 年 6 月 15 日

目录

1	前言及问题概述	2
2	相关基础知识及方法	2
2.1	相关原理	2
2.2	数据描述	3
2.3	特征工程与算法实现	5
3	实验结果与可扩展内容	6
3.1	探索性分析结果	6
3.2	模型比较结果	9
3.3	最终模型分析	12
3.4	参数调优及可扩展内容	13
4	讨论与结论	14
4.1	结果讨论	14
4.2	结论总结	14
5	心得体会与思考	15
5.1	个人心得	15
5.2	启发与思考	15
6	附录	16
	算法代码及其注释	16
	关键运行结果汇总	33

1 前言及问题概述

贷款审批是银行和互联网金融机构风险管理中的典型业务环节。审批人员需要综合申请人的收入、婚姻状况、教育背景、家庭负担、贷款金额、还款期限和历史信用记录等信息，判断其是否具备稳定的偿付能力。如果完全依赖人工审核，不仅处理速度较慢，而且不同审核人员可能对同一信息赋予不同权重。利用机器学习模型对历史审批数据进行学习，可以为审核人员提供一致、快速的辅助判断，并帮助识别影响审批结果的关键因素。

本次实验使用题目给出的 `loan.csv` 数据集。数据集共包含 614 条贷款申请记录和 13 个字段，目标变量为 `Loan_Status`，其中 Y 表示申请获得贷款，N 表示申请未获得贷款。题目图片将数据描述为“13 个特征”，但 CSV 的 13 列中还包括唯一申请编号 `Loan_ID` 和目标标签。因此，建模时删除不具有一般预测意义的申请编号，使用其余 11 个原始输入字段，并进一步构造总收入、贷款收入比和估计分期负担等组合特征。

题目要求利用集成方法训练分类器，预测申请者能否获得贷款。因此，本实验属于监督学习中的二分类任务。实验比较多数类基线、单棵决策树、Bagging、随机森林、极端随机树、AdaBoost、梯度提升和软投票集成模型，并使用准确率、宏平均 F1、平衡准确率、ROC-AUC、混淆矩阵和 5 折分层交叉验证综合评价模型。为了避免数据泄漏，缺失值填补、标准化和 One-Hot 编码均被封装在模型管线中，只使用当前训练折估计预处理参数。

2 相关基础知识及方法

2.1 相关原理

二分类模型根据输入特征向量 x 预测类别 $y \in \{N, Y\}$ ，可写为

$$\hat{y} = f(x), \quad (1)$$

其中 $\hat{y}$ 为预测的贷款审批结果。分类器还可以输出通过类别的概率 $P(y = Y | x)$ ，从而用于阈值调整、风险排序和 ROC 曲线分析。

单棵决策树通过递归划分特征空间形成一系列判断规则。设节点数据集为 D ，其中正负样本比例分别为 p 和 $1 - p$ ，其基尼不纯度为

$$Gini(D) = 1 - p^2 - (1 - p)^2. \quad (2)$$

决策树选择能够最大程度降低不纯度的特征和切分点。它具有直观、能够表达非线性关系等优点，但对训练数据扰动较敏感，单棵树容易产生较大方差。

Bagging 通过对训练集进行有放回抽样，训练 T 个基学习器，再对分类结果进行多数投票：

$$\hat{y} = \text{mode}\{f_1(x), f_2(x), \dots, f_T(x)\}. \quad (3)$$

随机森林在 Bagging 的基础上进一步对候选特征进行随机抽样，使不同树之间的相关性降低。极端随机树采用更强的随机切分策略，能够进一步降低方差，但随机性过强时也可能增加偏差。

Boosting 采用串行方式训练弱学习器，后续模型更加关注前面模型难以正确判断的样本。AdaBoost 根据分类错误动态调整样本权重；梯度提升则将新学习器拟合为损失函数负梯度的近似方向。其加法模型可表示为

$$F_M(x) = F_0(x) + \sum_{m=1}^M \eta h_m(x), \quad (4)$$

其中 $h_m(x)$ 为第 m 个弱学习器， η 为学习率。较小学习率配合较多弱学习器通常能够得到更平滑、稳定的模型。

软投票集成把多个不同类型分类器输出的类别概率进行加权平均：

$$P(y = Y | x) = \frac{\sum_{k=1}^K w_k P_k(y = Y | x)}{\sum_{k=1}^K w_k}, \quad (5)$$

最终选择平均概率较高的类别。本实验将逻辑回归、随机森林和梯度提升组合为软投票模型，使线性边界、随机树模型和序列提升模型互相补充。

本数据集的通过类约占 68.73%，类别分布并不均衡。若模型把所有样本都预测为 Y ，仍可得到约 0.69 的准确率，因此不能只依据 Accuracy 判断效果。本文同时使用宏平均 F1、平衡准确率和 ROC-AUC。对于正类，精确率、召回率和 F1 定义为

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}, \quad (6)$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}. \quad (7)$$

宏平均 F1 对两个类别的 F1 直接取平均，使未通过类与通过类具有相同权重；平衡准确率则是两个类别召回率的平均值。ROC-AUC 衡量模型将随机正类样本排在随机负类样本之前的概率，越接近 1 表示概率排序能力越强。

2.2 数据描述

原始数据共有 614 行、13 列，无完全重复记录。字段含义见表 1。Loan_ID 是每位申请人的唯一编号，在训练时删除；Loan_Status 是目标变量；其余字段作为原始输入变量。

表 1: 贷款审批数据集字段说明

字段	类型	含义
Loan_ID	标识字段	贷款申请编号，建模时删除

字段	类型	含义
Gender	类别特征	性别: Male 或 Female
Married	类别特征	是否已婚
Dependents	类别特征	抚养人数: 0、1、2 或 3+
Education	类别特征	是否为毕业生
Self_Employed	类别特征	是否为自由职业者
ApplicantIncome	数值特征	申请人收入
CoapplicantIncome	数值特征	共同申请人收入
LoanAmount	数值特征	贷款金额
Loan_Amount_Term	数值特征	还款期限
Credit_History	类别数值特征	是否具有合规信用历史
Property_Area	类别特征	房产所在地: 农村、城市或半城市
Loan_Status	目标变量	是否获得贷款: Y 或 N

数据集中共有 149 个缺失单元格, 具体分布见图 1。Credit_History 缺失 50 条, Self_Employed 缺失 32 条, LoanAmount 缺失 22 条, 其他字段也存在少量缺失。数值变量使用训练折中位数填补, 并增加缺失指示变量; 类别变量使用训练折众数填补。这种处理既保留全部样本, 也避免直接用全体数据统计量造成信息泄漏。

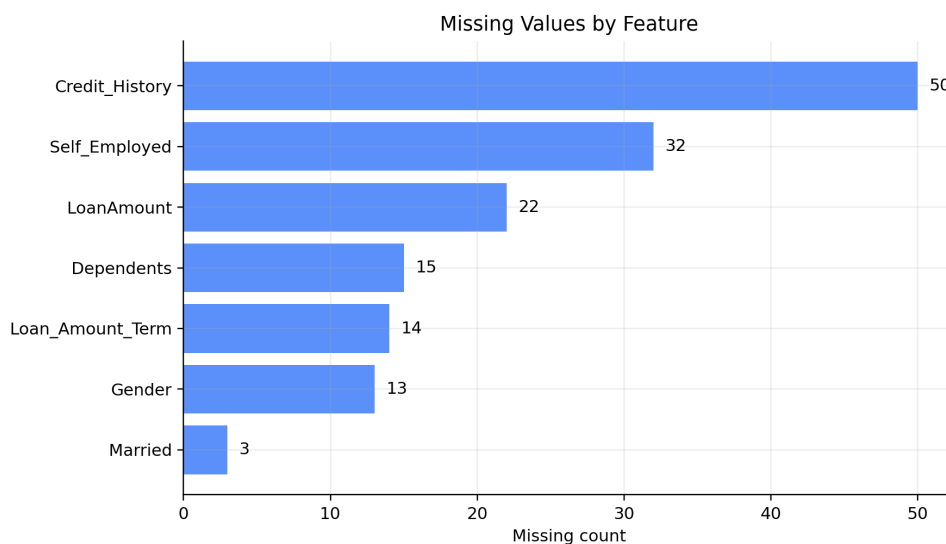


图 1: 各字段缺失值数量

目标变量分布见图 2。614 条记录中, 贷款通过 Y 有 422 条, 占 68.73%; 未通过 N 有 192 条, 占 31.27%。多数类基线在测试集上的准确率为 0.6911, 但其宏平均 F1 仅为 0.4087, 说明类别不均衡会显著影响仅看准确率时的判断。

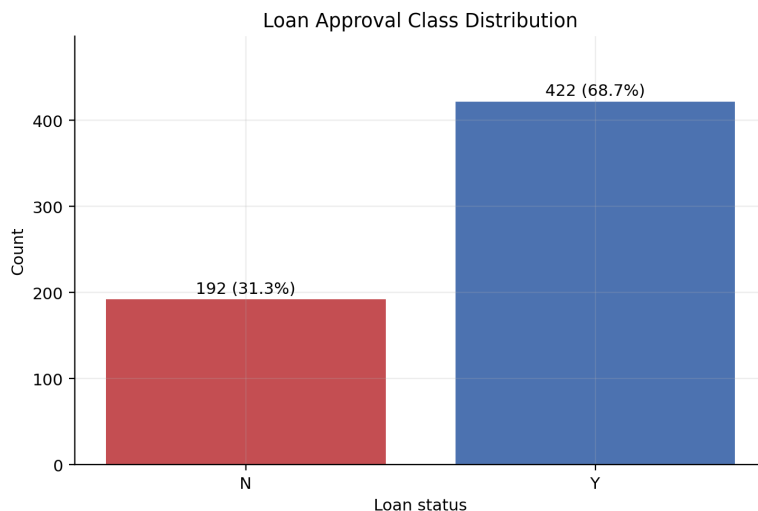


图 2: 贷款审批结果类别分布

2.3 特征工程与算法实现

原始收入和贷款金额具有明显右偏分布，而且贷款偿付能力通常取决于多个字段之间的比例关系。为此，本实验在原始变量基础上构造以下特征：

- **Dependents_Numeric**: 将 3+ 转换为 3，得到可计算的抚养人数。
- **TotalIncome**: 申请人与共同申请人收入之和。
- **LogTotalIncome** 和 **LogLoanAmount**: 对总收入和贷款金额进行对数变换，减弱极端值影响。
- **LoanIncomeRatio**: 贷款金额与总收入的比值，用于近似刻画贷款负担。
- **InstallmentProxy**: 贷款金额除以还款期限，作为每期偿还压力的近似量。
- **IncomePerDependent**: 总收入除以家庭负担人数加一，近似反映人均可支配收入。
- **HasCoapplicant**: 是否存在共同申请人收入。
- **CreditHistoryMissing**: 信用历史是否缺失，保留缺失本身可能包含的信息。

数值特征采用中位数填补和标准化，类别特征采用众数填补和 One-Hot 编码。数据按照 80%/20% 分层随机划分为 491 条训练样本和 123 条测试样本，固定随机种子为 42。训练集内部使用 5 折分层交叉验证。模型和主要参数见表 2。

表 2: 实验模型及主要参数

模型	集成方式	主要设置
DecisionTree	单模型对照	最大深度 5，叶节点至少 8 条
Bagging	并行集成	300 棵树，样本比例 0.85
RandomForest	并行集成	500 棵树，平衡子采样类别权重
ExtraTrees	强随机并行集成	500 棵树，叶节点至少 2 条
AdaBoost	串行提升	220 个浅树，学习率 0.035
GradientBoosting	梯度提升	250 棵浅树，学习率 0.03
SoftVoting	异构概率融合	逻辑回归、随机森林、梯度提升

程序实现流程如下：

1. 读取 `loan.csv`，检查数据规模、字段类型、重复记录、缺失值和目标分布。
2. 删除唯一编号 `Loan_ID`，构造收入、贷款负担和缺失指示特征。
3. 对数据进行 80%/20% 分层训练测试划分。
4. 使用 `ColumnTransformer` 分别处理数值变量和类别变量，并将预处理与分类器封装为 `Pipeline`。
5. 在训练集上进行 5 折分层交叉验证，计算 `Accuracy`、`Macro-F1`、`Balanced Accuracy` 和 `ROC-AUC`。
6. 在固定测试集上比较各模型，绘制模型对比图和 `ROC` 曲线。
7. 按训练集交叉验证 `Macro-F1` 选择最终模型，输出混淆矩阵、分类报告和置换重要性。

完整 Python 程序见附录。

3 实验结果与可扩展内容

3.1 探索性分析结果

图 3 展示了主要数值变量在贷款通过和未通过两组中的分布。两组申请人收入均存在较大离散程度，仅依靠收入绝对值难以形成清晰边界。未通过组平均贷款金额为 151.22，通过组为 144.29，差异存在但并不十分显著；这说明贷款结果更可能取决于贷款金额与收入、信用历史等变量的组合，而不是某个单独数值。

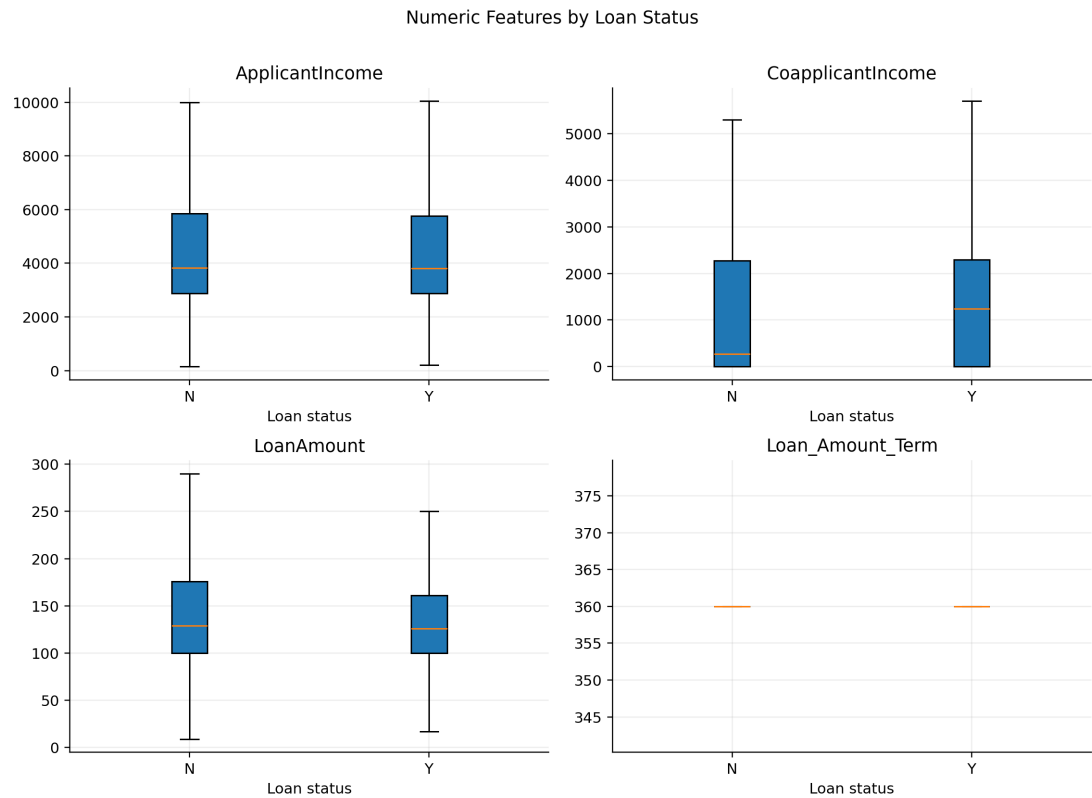


图 3: 主要数值特征在不同贷款结果下的分布

选定类别特征与审批通过率的关系见图 4。`Credit_History=1` 的申请人通过率为 79.58%，而 `Credit_History=0` 的申请人通过率仅为 7.87%，差距非常明显。毕业生通过率为 70.83%，非毕业生为 61.19%；半城市房产申请人通过率为 76.82%，高于城市的 65.84% 和农村的 61.45%。这些统计结果表明，信用历史是最重要的单一信号，教育和房产区域提供次要补充信息。

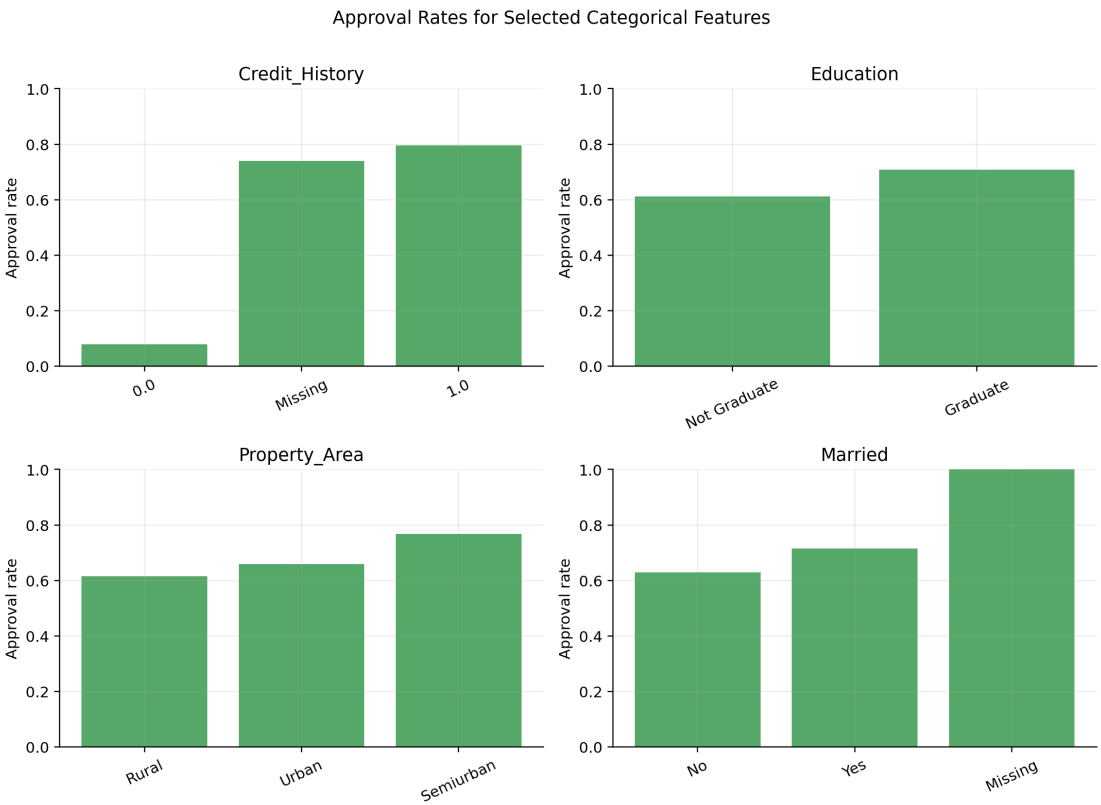


图 4: 部分类别特征对应的贷款通过率

数值特征相关性热力图见图 5。`ApplicantIncome` 与 `TotalIncome` 高度相关，`LoanAmount` 与对数贷款金额也高度相关，这是特征工程的预期结果。树模型可以从不同尺度和非线性切分中自动选择有效表达，因此保留这些变量用于比较；置换重要性则在最终解释时衡量每个原始或构造特征对泛化性能的实际贡献。

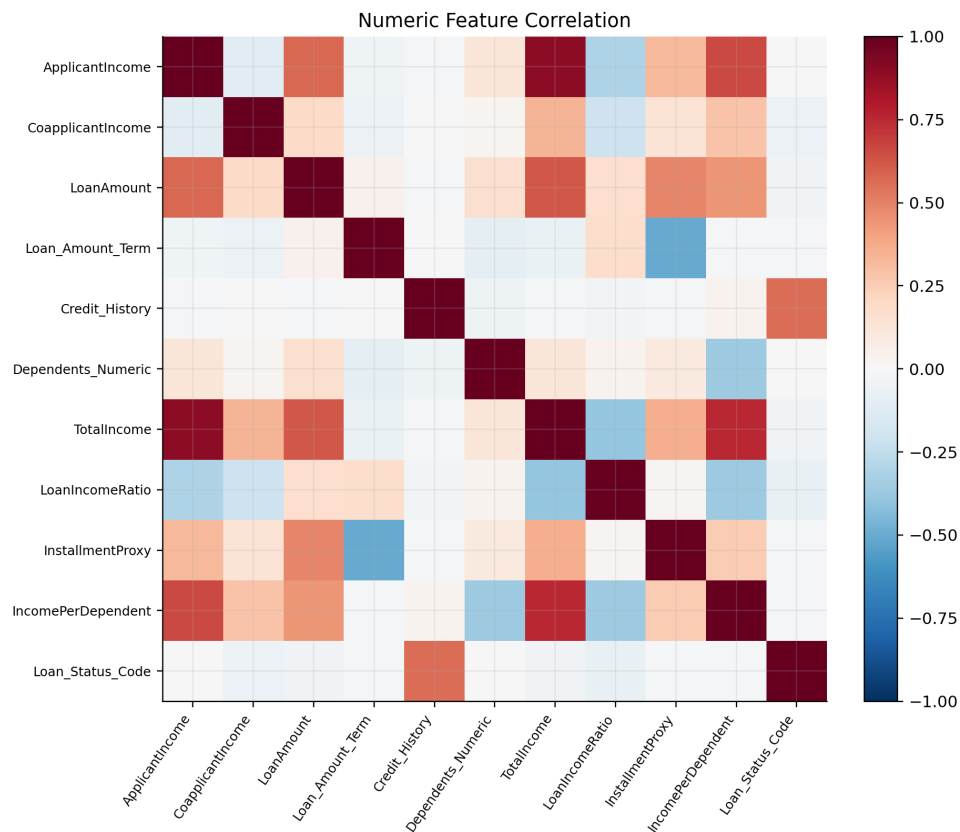


图 5: 数值变量及构造特征相关性热力图

3.2 模型比较结果

各模型测试集结果见表 3。软投票模型在本次固定测试集上表现最好，Accuracy 为 0.9024，Macro-F1 为 0.8802，ROC-AUC 为 0.8833。梯度提升测试集 Accuracy 为 0.8780，Macro-F1 为 0.8489。Bagging、随机森林和 AdaBoost 也明显优于多数类基线。单棵决策树 Macro-F1 为 0.8011，而多个集成模型达到 0.81 以上，说明组合多个基学习器能够改善总体分类性能。

表 3: 各模型测试集评价结果

模型	Accuracy	Bal. Acc.	Macro-F1	ROC-AUC
SoftVoting	0.9024	0.8639	0.8802	0.8833
GradientBoosting	0.8780	0.8317	0.8489	0.8808
Bagging	0.8699	0.8477	0.8477	0.8669
RandomForest	0.8537	0.8286	0.8286	0.8582
AdaBoost	0.8374	0.8096	0.8096	0.9203
DecisionTree	0.8211	0.8197	0.8011	0.8839
ExtraTrees	0.7967	0.7511	0.7565	0.8121
Majority_Baseline	0.6911	0.5000	0.4087	0.5000

图 6 同时给出测试集和交叉验证 Macro-F1。可以看到，软投票在测试集上的分数明显高于其交叉验证均值，说明本次测试划分可能较有利于该模型。如果仅根据测试集最高分选择模型，就会对一次随机划分产生过度依赖。因此，本实验预先规定按训练集 5 折交叉验证 Macro-F1 选择最终模型，测试集仅用于最终估计。

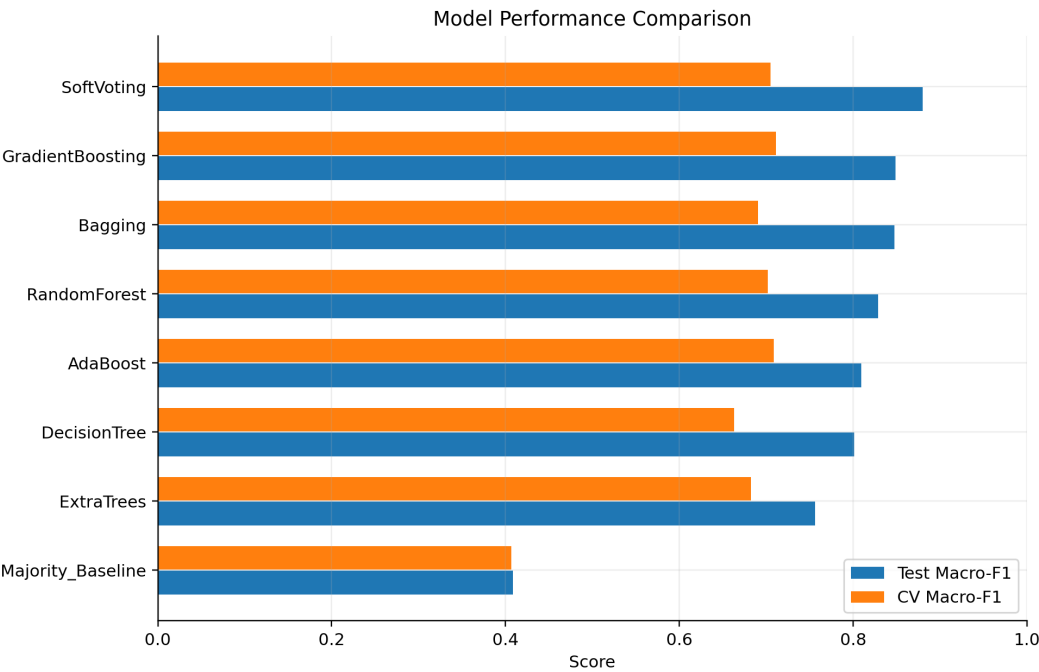


图 6: 各模型测试集与交叉验证 Macro-F1 对比

5 折交叉验证结果见表 4。梯度提升平均 Macro-F1 为 0.7115，略高于 AdaBoost 的 0.7088、软投票的 0.7049 和随机森林的 0.7021，因此被选为最终模型。前四个模型均值差距很小，而标准差约为 0.09–0.10，说明数据规模较小，不同折之间仍存在明显波动。较谨慎的结论应是：Boosting、软投票和随机森林都属于较可靠的候选方案，而不是某个模型具有压倒性优势。

表 4: 模型 5 折分层交叉验证结果

模型	CV Accuracy	CV Bal. Acc.	CV Macro-F1	CV ROC-AUC
GradientBoosting	0.7921	0.6954	0.7115	0.7740
AdaBoost	0.7860	0.6948	0.7088	0.7321
SoftVoting	0.7840	0.6913	0.7049	0.7784
RandomForest	0.7759	0.6925	0.7021	0.7716
Bagging	0.7657	0.6816	0.6907	0.7748
ExtraTrees	0.7413	0.6779	0.6827	0.7156
DecisionTree	0.7109	0.6642	0.6630	0.6994
Majority_Baseline	0.6864	0.5000	0.4070	0.5000

测试集 ROC 曲线见图 7。所有有效模型的 ROC-AUC 均明显高于随机分类器。AdaBoost 的测试集 ROC-AUC 最高，为 0.9203，但其默认 0.5 阈值下的 Macro-F1 为 0.8096。这说明“概率排序能力”和“固定阈值分类效果”并不完全相同。实际金融应用中可以根据误批和拒批成本重新调整阈值，而本报告为了公平比较，所有模型均使用默认分类规则。

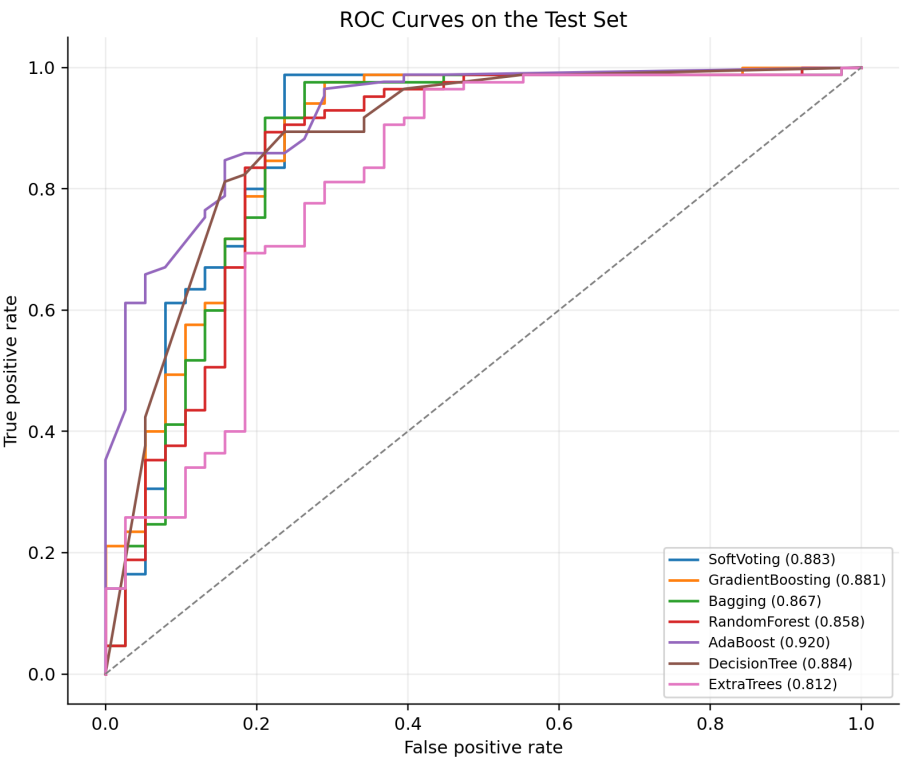


图 7: 各集成模型测试集 ROC 曲线

3.3 最终模型分析

按交叉验证选择的梯度提升模型在测试集上正确分类 108 条、错误分类 15 条，混淆矩阵见图 8。38 条实际未通过申请中，模型正确识别 27 条，误批 11 条；85 条实际通过申请中，模型正确识别 81 条，误拒 4 条。未通过类的 Precision 为 0.8710、Recall 为 0.7105、F1 为 0.7826；通过类的 Precision 为 0.8804、Recall 为 0.9529、F1 为 0.9153。

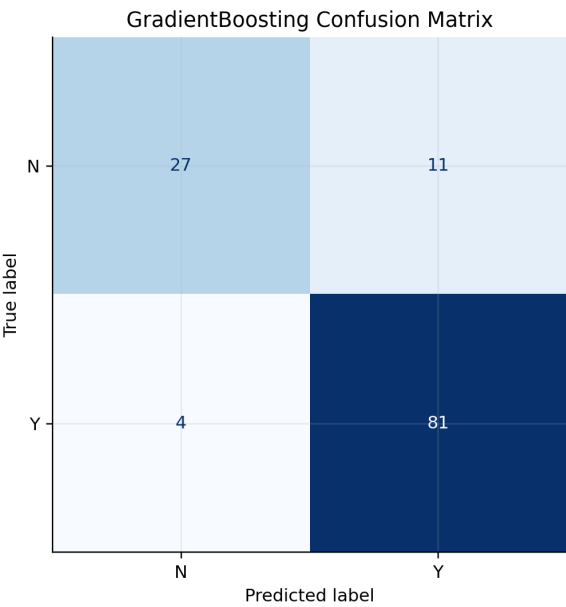


图 8: 梯度提升模型测试集混淆矩阵

模型对通过类的召回率较高，能够识别绝大多数历史上获得贷款的申请，但未通过类召回率相对较低。在实际风险控制场景中，误将高风险申请预测为通过通常比误拒低风险申请造成更大损失，因此可以适当提高通过阈值，或使用代价敏感学习对误批赋予更高惩罚。当前实验主要用于验证集成学习方法，不对业务成本进行主观设定。

置换重要性见图 9。Credit_History 的重要性远高于其他变量，随机打乱后测试集 Macro-F1 平均下降约 0.2652；LoanIncomeRatio 排名第二，下降约 0.0724；Property_Area 排名第三。该结果与探索性分析一致：历史信用是否合规是贷款审批的核心信号，而贷款金额相对于家庭总收入的负担比单独收入或单独贷款金额更有解释力。

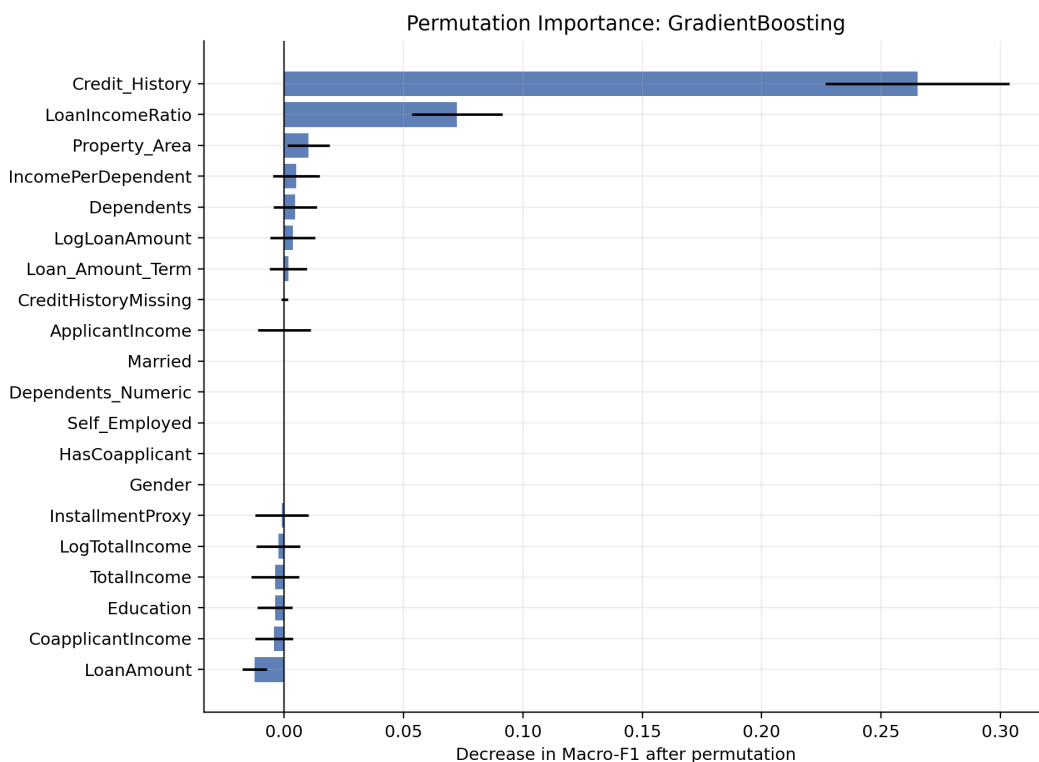


图 9: 梯度提升模型置换重要性

需要说明的是，置换重要性表示特征对当前模型预测性能的贡献，不等同于因果关系。例如，信用历史可能与其他未记录的经济条件相关，不能据此断言改变某个字段一定会直接改变审批结果。此外，性别、婚姻和教育等字段涉及公平性问题，即使在当前数据中贡献有限，真实部署前也应进行分组准确率、机会均等和差别影响审计。

3.4 参数调优及可扩展内容

本次实验采用一组合理固定参数完成模型对比，避免在较小数据集上进行过度搜索。后续可从以下方向扩展：

- 使用嵌套交叉验证调节梯度提升的树深、学习率、弱学习器数量和最小叶节点样本数，得到偏差更小的泛化估计。
- 使用重复分层交叉验证，降低单次 5 折划分造成的方差，并报告置信区间。
- 根据银行的误批与误拒成本设计代价矩阵，通过阈值搜索或类别权重优化业务目标，而不局限于默认 0.5 阈值。
- 尝试 XGBoost、LightGBM 或 CatBoost 等更成熟的提升算法，并使用早停控制过拟合。
- 对概率输出进行 Platt scaling 或等距回归校准，使预测概率更接近真实违约或审批频率。

- 按性别、婚姻、教育和房产区域分别计算召回率与误报率，检查模型是否对特定群体产生系统性不利影响。
- 收集负债、职业稳定性、资产、历史还款行为等更直接的偿付能力变量，提高预测效果和业务解释性。

4 讨论与结论

4.1 结果讨论

第一，集成学习整体优于简单基线。多数类模型的 Accuracy 看似达到 0.6911，但无法识别未通过类，Macro-F1 只有 0.4087。单棵决策树将测试集 Macro-F1 提高到 0.8011，而软投票、梯度提升、Bagging 和随机森林进一步达到 0.8286–0.8802，说明多学习器组合能够有效利用贷款申请特征。

第二，不同集成方式各有特点。Bagging 和随机森林通过平均多棵树降低方差，表现稳定；梯度提升和 AdaBoost 逐步修正错误样本，在交叉验证中位居前列；软投票能够融合线性和非线性模型，在固定测试集上取得最高分。但软投票测试分数与交叉验证均值之间存在较大差距，因此不能只看单次测试结果。

第三，信用历史是最关键的预测特征。具有合规信用历史的申请人通过率接近 80%，没有合规信用历史的申请人通过率不足 8%。置换重要性同样显示 `Credit_History` 对 Macro-F1 的影响远高于其他字段。贷款收入比位居第二，表明相对负担比收入或贷款金额的绝对值更能反映审批风险。

第四，模型仍然存在类别不对称。最终梯度提升模型对通过类的召回率为 0.9529，对未通过类的召回率为 0.7105。由于误批可能带来信用损失，实际应用应针对业务成本选择阈值，并由人工复核边界样本，而不能直接把模型输出作为最终审批决定。

4.2 结论总结

本次实验完成了基于贷款申请数据的集成分类任务。实验从数据读取开始，依次完成缺失值检查、探索性分析、特征工程、预处理管线构建、训练测试划分、5 折交叉验证、模型比较和结果解释。通过比较决策树、Bagging、随机森林、极端随机树、AdaBoost、梯度提升和软投票模型，验证了集成学习相对于多数类基线和单棵决策树的优势。

软投票模型在固定测试集上取得最高结果，Accuracy 为 0.9024，Macro-F1 为 0.8802；按训练集交叉验证 Macro-F1 选择的最终模型为梯度提升，其测试集 Accuracy 为 0.8780，Macro-F1 为 0.8489，ROC-AUC 为 0.8808。最终模型正确识别 27 条未通过申请和 81 条通过申请。综合分析表明，信用历史、贷款收入比和房产区域是当前数据中最有价值的预测信息。

总体而言，集成模型可以为贷款审批提供有效辅助，但 614 条样本仍不足以支持直接部署。模型结果应与业务规则、人工审核、公平性评估和概率校准结合使用。特别是涉及性别、婚姻和教育等敏感或代理变量时，需要确保模型不会把历史审批偏差固化为自动化决策。

5 心得体会与思考

5.1 个人心得

通过本次实验，我对 Bagging、随机森林、Boosting 和投票集成之间的差异有了更直观的理解。它们都叫作集成学习，但解决问题的方式不同：Bagging 更强调通过并行抽样降低方差，Boosting 更强调连续修正前一轮错误，投票集成则利用不同模型的互补性。只有同时观察交叉验证、测试集和混淆矩阵，才能判断某个高分究竟来自稳定能力还是一次数据划分的偶然性。

本次实验也让我进一步认识到预处理管线的重要性。如果先在完整数据集上填补缺失值或标准化，再做交叉验证，验证折的信息就会间接进入训练过程。把预处理和模型统一放入 Pipeline 后，每个训练折都独立估计填补值和编码规则，实验结果更可信，也更容易复现。

5.2 启发与思考

贷款审批不是只追求准确率的普通分类问题。不同错误的成本并不相同，误批可能造成资金损失，误拒则可能损害客户体验并错失业务机会。因此，真实系统应该先明确成本和监管要求，再选择评价指标与决策阈值。测试集上 AdaBoost 的 ROC-AUC 很高但默认阈值 F1 并非最高，正好说明概率排序和最终决策需要分开考虑。

此外，模型可解释性不能只停留在“哪个变量最重要”。信用历史的重要性很高，但模型预测仍然是相关性结果。对于性别、婚姻和教育等变量，还必须考虑公平性和合法合规问题。机器学习适合帮助审核人员组织信息、提示风险和提高效率，但不应取代对个人情况的综合判断，也不应成为无法申诉的黑箱决定。

6 附录

算法代码及其注释

```
from __future__ import annotations

import json
import warnings
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import (
    AdaBoostClassifier,
    BaggingClassifier,
    ExtraTreesClassifier,
    GradientBoostingClassifier,
    RandomForestClassifier,
    VotingClassifier,
)
from sklearn.impute import SimpleImputer
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    balanced_accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import StratifiedKFold, cross_validate,
↪ train_test_split
```

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier

warnings.filterwarnings("ignore", category=FutureWarning)

RANDOM_STATE = 42
TARGET = "Loan_Status"
ID_COLUMN = "Loan_ID"

def configure_plot_style() -> None:
    plt.rcParams.update(
        {
            "figure.dpi": 120,
            "axes.grid": True,
            "grid.alpha": 0.22,
            "axes.spines.top": False,
            "axes.spines.right": False,
            "font.size": 10,
        }
    )

def load_data(csv_path: Path) -> pd.DataFrame:
    df = pd.read_csv(csv_path)
    expected = {
        "Loan_ID",
        "Gender",
        "Married",
        "Dependents",
        "Education",
        "Self_Employed",
        "ApplicantIncome",
        "CoapplicantIncome",
        "LoanAmount",
        "Loan_Amount_Term",
        "Credit_History",
        "Property_Area",
        "Loan_Status",
    }
```

```

    }
    missing_columns = expected.difference(df.columns)
    if missing_columns:
        raise ValueError(f"Missing required columns:
        ↪ {sorted(missing_columns)}")
    return df

def engineer_features(raw_df: pd.DataFrame) -> pd.DataFrame:
    df = raw_df.drop(columns=[ID_COLUMN]).copy()

    dependents_numeric = (
        df["Dependents"].replace({"3+": "3"}).pipe(pd.to_numeric,
        ↪ errors="coerce")
    )
    total_income = df["ApplicantIncome"] + df["CoapplicantIncome"]
    loan_amount = df["LoanAmount"]
    loan_term = df["Loan_Amount_Term"]

    df["Dependents_Numeric"] = dependents_numeric
    df["TotalIncome"] = total_income
    df["LogTotalIncome"] = np.log1p(total_income)
    df["LogLoanAmount"] = np.log1p(loan_amount)
    df["LoanIncomeRatio"] = loan_amount / total_income.replace(0, np.nan)
    df["InstallmentProxy"] = loan_amount / loan_term.replace(0, np.nan)
    df["IncomePerDependent"] = total_income / (dependents_numeric.fillna(0) +
    ↪ 1)
    df["HasCoapplicant"] = np.where(df["CoapplicantIncome"] > 0, "Yes", "No")
    df["CreditHistoryMissing"] = np.where(df["Credit_History"].isna(),
    ↪ "Yes", "No")
    return df

def make_preprocessor(X: pd.DataFrame) -> tuple[ColumnTransformer,
    ↪ list[str], list[str]]:
    numeric_columns = X.select_dtypes(include=[np.number]).columns.tolist()
    categorical_columns = [column for column in X.columns if column not in
    ↪ numeric_columns]

    numeric_pipeline = Pipeline(
        steps=[

```

```

        ("imputer", SimpleImputer(strategy="median",
        ↪ add_indicator=True)),
        ("scaler", StandardScaler()),
    ]
)
categorical_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        (
            "onehot",
            OneHotEncoder(handle_unknown="ignore", sparse_output=False),
        ),
    ]
)
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_pipeline, numeric_columns),
        ("cat", categorical_pipeline, categorical_columns),
    ],
    remainder="drop",
)
return preprocessor, numeric_columns, categorical_columns

def build_models(preprocessor: ColumnTransformer) -> dict[str, Pipeline]:
    shallow_tree = DecisionTreeClassifier(
        max_depth=3,
        min_samples_leaf=8,
        class_weight="balanced",
        random_state=RANDOM_STATE,
    )
    voting_estimators = [
        (
            "lr",
            LogisticRegression(
                C=0.8,
                class_weight="balanced",
                max_iter=3000,
                random_state=RANDOM_STATE,
            ),
        ),
    ],

```

```

(
    "rf",
    RandomForestClassifier(
        n_estimators=350,
        min_samples_leaf=2,
        max_features="sqrt",
        class_weight="balanced_subsample",
        random_state=RANDOM_STATE,
        n_jobs=-1,
    ),
),
(
    "gb",
    GradientBoostingClassifier(
        n_estimators=180,
        learning_rate=0.035,
        max_depth=2,
        min_samples_leaf=8,
        subsample=0.85,
        random_state=RANDOM_STATE,
    ),
),
]

estimators = {
    "Majority_Baseline": DummyClassifier(strategy="most_frequent"),
    "DecisionTree": DecisionTreeClassifier(
        max_depth=5,
        min_samples_leaf=8,
        class_weight="balanced",
        random_state=RANDOM_STATE,
    ),
    "Bagging": BaggingClassifier(
        estimator=DecisionTreeClassifier(
            min_samples_leaf=4,
            class_weight="balanced",
            random_state=RANDOM_STATE,
        ),
        n_estimators=300,
        max_samples=0.85,
        max_features=0.9,
    ),
}

```

```
        bootstrap=True,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    ),
    "RandomForest": RandomForestClassifier(
        n_estimators=500,
        min_samples_leaf=2,
        max_features="sqrt",
        class_weight="balanced_subsample",
        random_state=RANDOM_STATE,
        n_jobs=-1,
    ),
    "ExtraTrees": ExtraTreesClassifier(
        n_estimators=500,
        min_samples_leaf=2,
        max_features="sqrt",
        class_weight="balanced",
        random_state=RANDOM_STATE,
        n_jobs=-1,
    ),
    "AdaBoost": AdaBoostClassifier(
        estimator=clone(shallow_tree),
        n_estimators=220,
        learning_rate=0.035,
        algorithm="SAMME",
        random_state=RANDOM_STATE,
    ),
    "GradientBoosting": GradientBoostingClassifier(
        n_estimators=250,
        learning_rate=0.03,
        max_depth=2,
        min_samples_leaf=8,
        subsample=0.85,
        random_state=RANDOM_STATE,
    ),
    "SoftVoting": VotingClassifier(
        estimators=voting_estimators,
        voting="soft",
        weights=[1, 2, 2],
        n_jobs=-1,
    ),
```

```

    }
    return {
        name: Pipeline(
            steps=[("preprocess", clone(preprocessor)), ("model", estimator)]
        )
        for name, estimator in estimators.items()
    }

def save_data_overview(raw_df: pd.DataFrame, output_dir: Path) -> None:
    overview_dir = output_dir / "01_data_overview"
    overview_dir.mkdir(parents=True, exist_ok=True)
    raw_df.head(10).to_csv(overview_dir / "first_10_rows.csv", index=False,
        ↪ encoding="utf-8-sig")
    raw_df.dtypes.astype(str).rename("dtype").to_csv(
        overview_dir / "column_types.csv", encoding="utf-8-sig"
    )
    raw_df.isna().sum().rename("missing_count").to_csv(
        overview_dir / "missing_values.csv", encoding="utf-8-sig"
    )
    raw_df[TARGET].value_counts().rename("count").to_csv(
        overview_dir / "target_distribution.csv", encoding="utf-8-sig"
    )
    raw_df.describe(include="all").transpose().to_csv(
        overview_dir / "descriptive_statistics.csv", encoding="utf-8-sig"
    )

def plot_exploratory_figures(raw_df: pd.DataFrame, figure_dir: Path,
    ↪ output_dir: Path) -> None:
    figure_dir.mkdir(parents=True, exist_ok=True)
    y_order = ["N", "Y"]
    colors = ["#C44E52", "#4C72B0"]

    missing = raw_df.isna().sum().sort_values(ascending=True)
    missing = missing[missing > 0]
    fig, ax = plt.subplots(figsize=(8.2, 4.7))
    ax.barh(missing.index, missing.values, color="#5B8FF9")
    for index, value in enumerate(missing.values):
        ax.text(value + 0.8, index, str(value), va="center")
    ax.set_xlabel("Missing count")

```

```

ax.set_title("Missing Values by Feature")
fig.tight_layout()
fig.savefig(figure_dir / "missing_values.png", dpi=220)
plt.close(fig)

counts = raw_df[TARGET].value_counts().reindex(y_order)
fig, ax = plt.subplots(figsize=(6.8, 4.6))
bars = ax.bar(counts.index, counts.values, color=colors)
for bar, value in zip(bars, counts.values):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        value + 7,
        f"{value} ({value / len(raw_df):.1%})",
        ha="center",
    )
ax.set_ylim(0, counts.max() * 1.18)
ax.set_xlabel("Loan status")
ax.set_ylabel("Count")
ax.set_title("Loan Approval Class Distribution")
fig.tight_layout()
fig.savefig(figure_dir / "target_distribution.png", dpi=220)
plt.close(fig)

numeric_features = [
    "ApplicantIncome",
    "CoapplicantIncome",
    "LoanAmount",
    "Loan_Amount_Term",
]

fig, axes = plt.subplots(2, 2, figsize=(10.5, 7.5))
for ax, feature in zip(axes.flat, numeric_features):
    values = [raw_df.loc[raw_df[TARGET] == label, feature].dropna() for
    ↪ label in y_order]
    ax.boxplot(values, tick_labels=y_order, showfliers=False,
    ↪ patch_artist=True)
    for patch, color in zip(ax.artists, colors):
        patch.set_facecolor(color)
    ax.set_title(feature)
    ax.set_xlabel("Loan status")
fig.suptitle("Numeric Features by Loan Status", y=1.01)
fig.tight_layout()

```

```

fig.savefig(ffigure_dir / "numeric_features_by_status.png", dpi=220,
↪  bbox_inches="tight")
plt.close(fig)

category_features = ["Credit_History", "Education", "Property_Area",
↪  "Married"]
fig, axes = plt.subplots(2, 2, figsize=(10.5, 7.5))
for ax, feature in zip(axes.flat, category_features):
    temp = raw_df[[feature, TARGET]].copy()
    temp[feature] = temp[feature].fillna("Missing").astype(str)
    approval = temp.groupby(feature)[TARGET].apply(lambda series:
↪  (series == "Y").mean())
    approval = approval.sort_values()
    ax.bar(approval.index, approval.values, color="#55A868")
    ax.set_ylim(0, 1)
    ax.set_title(feature)
    ax.set_ylabel("Approval rate")
    ax.tick_params(axis="x", rotation=25)
fig.suptitle("Approval Rates for Selected Categorical Features", y=1.01)
fig.tight_layout()
fig.savefig(ffigure_dir / "categorical_approval_rates.png", dpi=220,
↪  bbox_inches="tight")
plt.close(fig)

engineered = engineer_features(raw_df)
correlation_columns = [
    "ApplicantIncome",
    "CoapplicantIncome",
    "LoanAmount",
    "Loan_Amount_Term",
    "Credit_History",
    "Dependents_Numeric",
    "TotalIncome",
    "LoanIncomeRatio",
    "InstallmentProxy",
    "IncomePerDependent",
]
corr_df = engineered[correlation_columns + [TARGET]].copy()
corr_df["Loan_Status_Code"] = corr_df[TARGET].map({"N": 0, "Y": 1})
corr = corr_df[correlation_columns + ["Loan_Status_Code"]].corr()

```

```

corr.to_csv(output_dir / "01_data_overview" / "numeric_correlation.csv",
            ↪ encoding="utf-8-sig")
fig, ax = plt.subplots(figsize=(9.3, 7.4))
image = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
ax.set_xticks(np.arange(len(corr.columns)))
ax.set_yticks(np.arange(len(corr.index)))
ax.set_xticklabels(corr.columns, rotation=55, ha="right", fontsize=8)
ax.set_yticklabels(corr.index, fontsize=8)
ax.set_title("Numeric Feature Correlation")
fig.colorbar(image, ax=ax, fraction=0.046, pad=0.04)
fig.tight_layout()
fig.savefig(figure_dir / "correlation_heatmap.png", dpi=220)
plt.close(fig)

def evaluate_models(
    models: dict[str, Pipeline],
    X_train: pd.DataFrame,
    X_test: pd.DataFrame,
    y_train: pd.Series,
    y_test: pd.Series,
    output_dir: Path,
) -> tuple[pd.DataFrame, pd.DataFrame, dict[str, Pipeline]]:
    model_dir = output_dir / "02_model_results"
    model_dir.mkdir(parents=True, exist_ok=True)
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    scoring = {
        "accuracy": "accuracy",
        "balanced_accuracy": "balanced_accuracy",
        "precision_macro": "precision_macro",
        "recall_macro": "recall_macro",
        "f1_macro": "f1_macro",
        "roc_auc": "roc_auc",
    }
    test_rows: list[dict[str, float | str]] = []
    cv_rows: list[dict[str, float | str]] = []
    fitted_models: dict[str, Pipeline] = {}

    for name, pipeline in models.items():
        cv_result = cross_validate(
            pipeline,

```

```

        X_train,
        y_train,
        cv=cv,
        scoring=scoring,
        n_jobs=-1,
        error_score="raise",
    )
    cv_row: dict[str, float | str] = {"model": name}
    for metric in scoring:
        values = cv_result[f"test_{metric}"]
        cv_row[f"cv_{metric}_mean"] = float(values.mean())
        cv_row[f"cv_{metric}_std"] = float(values.std(ddof=1))
    cv_rows.append(cv_row)

    fitted = clone(pipeline).fit(X_train, y_train)
    fitted_models[name] = fitted
    prediction = fitted.predict(X_test)
    probability = fitted.predict_proba(X_test)[:,
↪ list(fitted.classes_).index("Y")]
    test_rows.append(
        {
            "model": name,
            "accuracy": accuracy_score(y_test, prediction),
            "balanced_accuracy": balanced_accuracy_score(y_test,
↪ prediction),
            "precision_macro": precision_score(y_test, prediction,
↪ average="macro", zero_division=0),
            "recall_macro": recall_score(y_test, prediction,
↪ average="macro", zero_division=0),
            "f1_macro": f1_score(y_test, prediction, average="macro",
↪ zero_division=0),
            "roc_auc": roc_auc_score((y_test == "Y").astype(int),
↪ probability),
        }
    )

test_results = pd.DataFrame(test_rows).sort_values("f1_macro",
↪ ascending=False)
cv_results = pd.DataFrame(cv_rows).sort_values("cv_f1_macro_mean",
↪ ascending=False)

```

```

test_results.to_csv(model_dir / "test_metrics.csv", index=False,
    ↪ encoding="utf-8-sig")
cv_results.to_csv(model_dir / "cross_validation_metrics.csv",
    ↪ index=False, encoding="utf-8-sig")
return test_results, cv_results, fitted_models

def plot_model_results(
    test_results: pd.DataFrame,
    cv_results: pd.DataFrame,
    fitted_models: dict[str, Pipeline],
    X_test: pd.DataFrame,
    y_test: pd.Series,
    figure_dir: Path,
    output_dir: Path,
) -> tuple[str, dict[str, object]]:
    merged = test_results.merge(
        cv_results[["model", "cv_f1_macro_mean", "cv_f1_macro_std"]],
        ↪ on="model"
    ).sort_values("f1_macro")
    y_positions = np.arange(len(merged))
    fig, ax = plt.subplots(figsize=(9.0, 5.8))
    ax.barh(y_positions - 0.18, merged["f1_macro"], height=0.34, label="Test
    ↪ Macro-F1")
    ax.barh(y_positions + 0.18, merged["cv_f1_macro_mean"], height=0.34,
    ↪ label="CV Macro-F1")
    ax.set_yticks(y_positions)
    ax.set_yticklabels(merged["model"])
    ax.set_xlim(0, 1)
    ax.set_xlabel("Score")
    ax.set_title("Model Performance Comparison")
    ax.legend(loc="lower right")
    fig.tight_layout()
    fig.savefig(figure_dir / "model_comparison.png", dpi=220)
    plt.close(fig)

    fig, ax = plt.subplots(figsize=(7.4, 6.2))
    for name in test_results["model"]:
        if name == "Majority_Baseline":
            continue
        model = fitted_models[name]

```

```

probability = model.predict_proba(X_test)[: ,
    ↪ list(model.classes_).index("Y")]
false_positive_rate, true_positive_rate, _ = roc_curve(
    (y_test == "Y").astype(int), probability
)
auc_value = roc_auc_score((y_test == "Y").astype(int), probability)
ax.plot(false_positive_rate, true_positive_rate, label=f"{name}"
    ↪ ({auc_value:.3f}))
ax.plot([0, 1], [0, 1], linestyle="--", color="gray", linewidth=1)
ax.set_xlabel("False positive rate")
ax.set_ylabel("True positive rate")
ax.set_title("ROC Curves on the Test Set")
ax.legend(fontsize=8, loc="lower right")
fig.tight_layout()
fig.savefig(figure_dir / "roc_curves.png", dpi=220)
plt.close(fig)

candidate_cv = cv_results[cv_results["model"] != "Majority_Baseline"]
best_name = str(candidate_cv.iloc[0]["model"])
best_model = fitted_models[best_name]
best_prediction = best_model.predict(X_test)
best_probability = best_model.predict_proba(X_test)[: ,
    ↪ list(best_model.classes_).index("Y")]

matrix = confusion_matrix(y_test, best_prediction, labels=["N", "Y"])
fig, ax = plt.subplots(figsize=(5.8, 5.0))
display = ConfusionMatrixDisplay(confusion_matrix=matrix,
    ↪ display_labels=["N", "Y"])
display.plot(ax=ax, cmap="Blues", colorbar=False)
ax.set_title(f"{best_name} Confusion Matrix")
fig.tight_layout()
fig.savefig(figure_dir / "best_confusion_matrix.png", dpi=220)
plt.close(fig)

importance = permutation_importance(
    best_model,
    X_test,
    y_test,
    scoring="f1_macro",
    n_repeats=40,
    random_state=RANDOM_STATE,

```

```

        n_jobs=-1,
    )
    importance_table = pd.DataFrame(
        {
            "feature": X_test.columns,
            "importance_mean": importance.importances_mean,
            "importance_std": importance.importances_std,
        }
    ).sort_values("importance_mean", ascending=False)
    importance_table.to_csv(
        output_dir / "02_model_results" / "permutation_importance.csv",
        index=False,
        encoding="utf-8-sig",
    )

    plot_importance =
    ↪ importance_table.head(20).sort_values("importance_mean")
    fig, ax = plt.subplots(figsize=(8.8, 6.4))
    ax.barh(
        plot_importance["feature"],
        plot_importance["importance_mean"],
        xerr=plot_importance["importance_std"],
        color="#4C72B0",
        alpha=0.9,
    )
    ax.axvline(0, color="black", linewidth=0.8)
    ax.set_xlabel("Decrease in Macro-F1 after permutation")
    ax.set_title(f"Permutation Importance: {best_name}")
    fig.tight_layout()
    fig.savefig(output_dir / "feature_importance_top20.png", dpi=220)
    plt.close(fig)

    report = classification_report(y_test, best_prediction,
    ↪ output_dict=True, zero_division=0)
    pd.DataFrame(report).transpose().to_csv(
        output_dir / "02_model_results" / "best_classification_report.csv",
        encoding="utf-8-sig",
    )

    pd.DataFrame(matrix, index=["actual_N", "actual_Y"], columns=["pred_N",
    ↪ "pred_Y"]).to_csv(
        output_dir / "02_model_results" / "best_confusion_matrix.csv",
        encoding="utf-8-sig",

```

```

    )

    summary = {
        "best_model": best_name,
        "confusion_matrix": matrix.tolist(),
        "classification_report": report,
        "test_roc_auc": float(roc_auc_score((y_test == "Y").astype(int),
        ↪ best_probability)),
        "top_features": importance_table.head(10).to_dict(orient="records"),
    }

    return best_name, summary

def main() -> None:
    figure_plot_style()
    base_dir = Path(__file__).resolve().parent
    data_path = base_dir / "loan.csv"
    figure_dir = base_dir / "figures"
    output_dir = base_dir / "results_loan_ensemble"
    output_dir.mkdir(parents=True, exist_ok=True)

    raw_df = load_data(data_path)
    save_data_overview(raw_df, output_dir)
    plot_exploratory_figures(raw_df, figure_dir, output_dir)

    model_df = engineer_features(raw_df)
    X = model_df.drop(columns=[TARGET])
    y = model_df[TARGET]
    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.20,
        stratify=y,
        random_state=RANDOM_STATE,
    )

    preprocessor, numeric_columns, categorical_columns = make_preprocessor(X)
    models = build_models(preprocessor)
    test_results, cv_results, fitted_models = evaluate_models(
        models, X_train, X_test, y_train, y_test, output_dir
    )

```

```

best_name, best_summary = plot_model_results(
    test_results,
    cv_results,
    fitted_models,
    X_test,
    y_test,
    figure_dir,
    output_dir,
)

summary = {
    "data_shape": list(raw_df.shape),
    "missing_cells": int(raw_df.isna().sum().sum()),
    "duplicate_rows": int(raw_df.duplicated().sum()),
    "target_counts": raw_df[TARGET].value_counts().to_dict(),
    "train_size": int(len(X_train)),
    "test_size": int(len(X_test)),
    "numeric_columns": numeric_columns,
    "categorical_columns": categorical_columns,
    "best_model": best_name,
    "best_model_details": best_summary,
}

with (output_dir / "summary.json").open("w", encoding="utf-8") as file:
    json.dump(summary, file, ensure_ascii=False, indent=2)

print(f"Data shape: {raw_df.shape}")
print(f"Missing cells: {raw_df.isna().sum().sum()}")
print(f"Target counts: {raw_df[TARGET].value_counts().to_dict()}")
print(f"Train/test sizes: {len(X_train)}/{len(X_test)}")
print("\nTest metrics:")
print(test_results.to_string(index=False, float_format=lambda value:
    ↪ f"{value:.4f}"))
print("\nCross-validation metrics:")
print(
    cv_results[
        [
            "model",
            "cv_accuracy_mean",
            "cv_balanced_accuracy_mean",
            "cv_f1_macro_mean",
            "cv_roc_auc_mean",

```

```
        ]
    ].to_string(index=False, float_format=lambda value: f"{value:.4f}")
)
print(f"\nBest model selected by CV Macro-F1: {best_name}")

if __name__ == "__main__":
    main()
```

关键运行结果汇总

数据规模: (614, 13)	
原始输入字段: 11 个 (删除 Loan_ID, Loan_Status 为目标)	
缺失单元格: 149	
训练集样本数: 491	
测试集样本数: 123	
类别分布:	
Y: 422 (68.73%)	
N: 192 (31.27%)	
测试集模型结果:	
SoftVoting	Accuracy=0.9024, Macro-F1=0.8802, ROC-AUC=0.8833
GradientBoosting	Accuracy=0.8780, Macro-F1=0.8489, ROC-AUC=0.8808
Bagging	Accuracy=0.8699, Macro-F1=0.8477, ROC-AUC=0.8669
RandomForest	Accuracy=0.8537, Macro-F1=0.8286, ROC-AUC=0.8582
AdaBoost	Accuracy=0.8374, Macro-F1=0.8096, ROC-AUC=0.9203
DecisionTree	Accuracy=0.8211, Macro-F1=0.8011, ROC-AUC=0.8839
ExtraTrees	Accuracy=0.7967, Macro-F1=0.7565, ROC-AUC=0.8121
Majority_Baseline	Accuracy=0.6911, Macro-F1=0.4087, ROC-AUC=0.5000
训练集 5 折交叉验证 Macro-F1:	
GradientBoosting=0.7115	
AdaBoost=0.7088	
SoftVoting=0.7049	
RandomForest=0.7021	
Bagging=0.6907	
ExtraTrees=0.6827	
DecisionTree=0.6630	
Majority_Baseline=0.4070	
最终模型: GradientBoosting (按交叉验证 Macro-F1 选择)	
测试集分类报告:	
N: precision=0.8710, recall=0.7105, f1-score=0.7826, support=38	
Y: precision=0.8804, recall=0.9529, f1-score=0.9153, support=85	
混淆矩阵:	
	预测 N 预测 Y
真实 N	27 11
真实 Y	4 81
置换重要性前三:	
Credit_History=0.2652	
LoanIncomeRatio=0.0724	
Property_Area=0.0102	